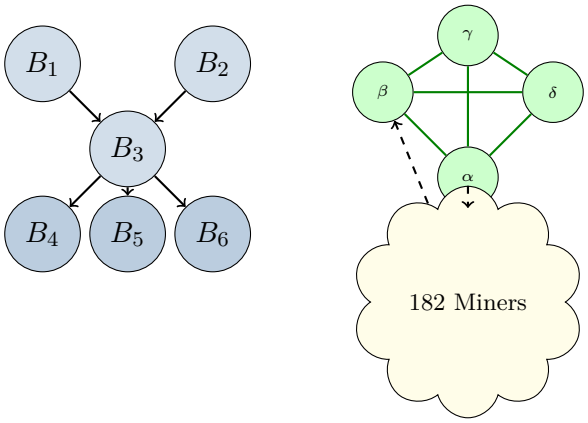
Q-NarwhalKnight

A Quantum-Resistant Peer-to-Peer Network Architecture

Technical Whitepaper v2.0

Mainnet Edition — v8.2.6



Q-NarwhalKnight Development Team

February 2026

Mainnet Launch: February 22, 2026

Abstract

Q-NarwhalKnight is a production-deployed, quantum-resistant peer-to-peer network operating on mainnet since February 2026. Built on libp2p-rust, the system combines DAG-Knight consensus with Narwhal mempool technology and a novel **Apollo Sync** optimization engine that employs Kalman filtering, PID rate control, and gravitational peer modeling for adaptive synchronization. This paper describes the complete network architecture as deployed at v8.2.6, covering the four-server high-availability infrastructure, zero-downtime rolling deployments, Dandelion++ gossip privacy, the PI-controlled emission system maintaining 2,625,000 QUG/year issuance, and critical safety systems including sync-down protection, immediate balance persistence, and height-gated consensus upgrades. As of this writing, the network sustains 182 active miners producing 458 MH/s aggregate hashpower, with 394 registered wallets and 8,267 QUG mined of the 21 million maximum supply.

Keywords: Blockchain, DAG Consensus, libp2p, Post-Quantum Cryptography, Peer-to-Peer Networks, Byzantine Fault Tolerance, Kalman Filter Sync, PI Emission Control, High Availability

Contents

1 Introduction

Modern blockchain networks face three critical challenges: **scalability**, **latency**, and **quantum resistance**. Traditional linear blockchains suffer from inherent throughput limitations, while naive parallelization approaches compromise security guarantees. Q-NarwhalKnight addresses these challenges through a carefully designed network architecture that leverages:

1. **DAG-Knight Consensus**: A directed acyclic graph (DAG) based consensus protocol that achieves zero-message-complexity Byzantine fault tolerance
2. **Narwhal Mempool**: A reliable broadcast primitive that separates data dissemination from consensus ordering
3. **libp2p-rust**: A modular networking stack providing transport-agnostic peer-to-peer communication
4. **Post-Quantum Cryptography**: Integration of Dilithium5 signatures and Kyber1024 key encapsulation with crypto-agile height-gated transitions
5. **Apollo Sync Engine**: A control-theoretic synchronization optimizer using Kalman filtering, PID rate control, and gravitational peer modeling
6. **PI Emission Controller**: A proportional-integral feedback controller that maintains target emission rates across varying network conditions

This paper describes the complete network layer as deployed on the Q-NarwhalKnight mainnet (network ID: `mainnet-genesis`), which has been in continuous operation since February 22, 2026.

1.1 Document History

Version	Date	Changes
v1.0	Dec 2025	Initial whitepaper (testnet architecture)
v2.0	Feb 2026	Mainnet edition: Apollo Sync, HA deployment, PI emission, Dandelion++, balance persistence, 4-server infrastructure, production metrics

2 Network Architecture Overview

2.1 Unified Network Manager

The Q-NarwhalKnight network layer is orchestrated by the `UnifiedNetworkManager`, a central component that coordinates all peer-to-peer operations. Unlike traditional blockchain clients that treat networking as a simple message-passing layer, Q-NarwhalKnight's network manager actively participates in consensus decisions and adaptive synchronization through the Apollo control systems.

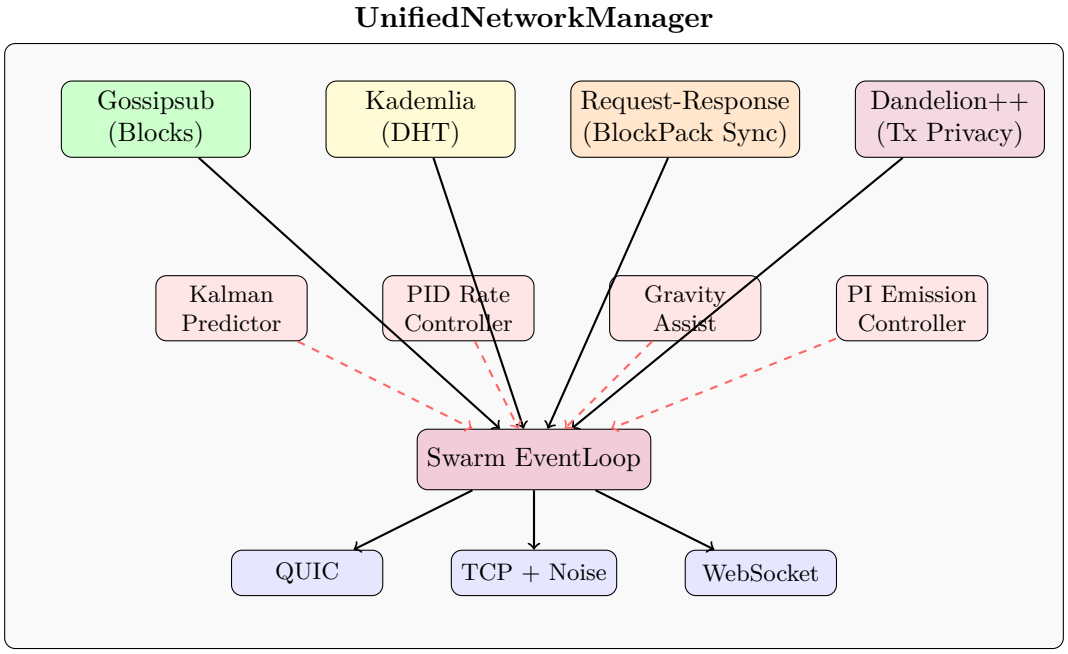


Figure 1: Unified Network Manager Architecture (v8.2.6)

2.2 Transport Layer

Q-NarwhalKnight supports multiple transport protocols to maximize connectivity:

- **QUIC** (Primary): Provides multiplexed streams with built-in encryption via TLS 1.3, reducing connection overhead for high-frequency block gossip
- **TCP + Noise**: Fallback transport using Noise Protocol Framework for encryption, used when QUIC is blocked by network firewalls
- **WebSocket**: Enables browser-based light clients to participate in the network via the Quillon wallet web interface

The libp2p `Swarm` abstraction allows seamless protocol negotiation, automatically selecting the optimal transport based on peer capabilities.

2.3 Production Infrastructure

The mainnet operates on a four-server high-availability infrastructure:

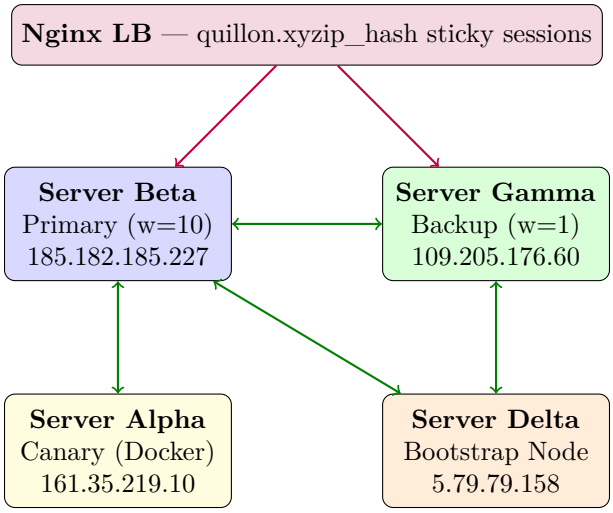


Figure 2: Four-Server High-Availability Infrastructure

Server	Role	Ports	Special Configuration
Beta	Primary bootstrap	8080/9001	16GB RAM, RocksDB cache 4GB
Gamma	Hot standby	8080/9001	8GB RAM + 4GB swap, cache 512MB
Alpha	Canary testing	Docker	Debian 12 container on Debian 11
Delta	4th bootstrap	8080/9001	RocksDB cache 2GB

Table 1: Server Configuration Summary

3 Protocol Stack

3.1 Gossipsub: Block Propagation

Q-NarwhalKnight uses libp2p's Gossipsub v1.1 for block propagation, with custom topic configuration optimized for DAG-Knight's parallel block production.

3.1.1 Topic Structure

Topic	Purpose
`/qnk/{network_id}/blocks`	Full block announcements via DAG gossipsub
`/qnk/{network_id}/peer-heights`	Height announcements for sync coordination
`/qnk/{network_id}/turbo-sync-request`	Batch sync request coordination
`/qnk/{network_id}/turbo-sync-response`	Batch sync response routing
`/qnk/{network_id}/balance-updates`	Balance state propagation (P2P consensus)
`/qnk/{network_id}/miner-stats`	Mining statistics broadcast
`/qnk/{network_id}/pool-*`	Mining pool coordination topics

Table 2: Gossipsub Topic Structure (v8.2.6)

3.1.2 Network ID Isolation

A critical safety property: gossipsub topics are namespaced by `network_id`, providing complete isolation between network generations. When a new mainnet epoch launches (e.g., `mainnet2026.1`

→ `mainnet-genesis`), nodes on different networks cannot exchange messages, blocks, or sync state. This is enforced at three layers:

1. **Gossipsub Topic Isolation**: Different topic strings prevent message exchange
2. **Protocol Handshake Validation**: Peers validate `network_id` before data exchange
3. **HTTP State Sync**: `FullStateSnapshot` embeds `network_id`—rejects mismatches

3.1.3 Message Validation

Unlike traditional blockchains where gossip validation is binary (valid/invalid), Q-NarwhalKnight implements a three-tier validation strategy:

1. **Syntax Validation**: Immediate rejection of malformed messages
2. **Semantic Validation**: Signature and hash verification
3. **Contextual Validation**: DAG parent verification (may defer if parents unknown)

```
fn validate_message(&self, msg: &GossipsubMessage) -> MessageAcceptance {
    // Tier 1: Syntax
    let block = match deserialize_block(&msg.data) {
        Ok(b) => b,
        Err(_) => return MessageAcceptance::Reject,
    };

    // Tier 2: Semantic (signature + hash)
    if !verify_block_signature(&block) {
        return MessageAcceptance::Reject;
    }

    // Tier 3: Contextual (DAG parents)
    if !self.has_all_parents(&block.dag_parents) {
        return MessageAcceptance::Ignore; // May receive parents later
    }

    MessageAcceptance::Accept
}
```

Listing 1: Three-tier message validation

3.2 Kademlia DHT: Peer Discovery

The Kademlia Distributed Hash Table serves dual purposes:

1. **Peer Discovery**: Finding nodes by their PeerId across the global network
2. **Content Routing**: Locating peers that have specific block ranges for targeted sync

3.2.1 Bootstrap Process with Connection Warmup

New nodes bootstrap through a combination of hardcoded bootstrap peer addresses and DNS-based peer discovery. Starting with v3.3.7, Q-NarwhalKnight implements **automatic connection warmup** for newly generated identities:

1. Connect to bootstrap peer(s) via hardcoded multiaddresses
2. Perform iterative `FIND_NODE` for random IDs to populate routing table

3. If identity is **new** (first boot): execute retry loop with exponential backoff (3s, 6s, 12s) re-triggering Kademlia DHT bootstrap after failures
4. Build routing table with k-bucket diversity
5. Subscribe to all gossipsub topics
6. Announce local height to trigger sync if behind

This warmup procedure resolves a class of first-boot failures where DHT routing tables are empty and bootstrap connections fail silently.

3.3 Request-Response: Block Synchronization

For bulk block synchronization, Q-NarwhalKnight implements a custom request-response protocol using libp2p's `request_response` behavior with CBOR encoding, LZ4 compression, and adaptive batching.

3.3.1 BlockPack Protocol

Field	Type	Description
Request (/qnk/blockpack/1.0.0)		
`start_height`	`u64`	Starting block height
`count`	`u32`	Number of blocks requested (adaptive)
`include_balance_updates`	`bool`	Include balance data with blocks
`network_id`	`String`	Network isolation identifier
Response		
`blocks`	`Vec<QBlock>`	LZ4-compressed block data
`balance_updates`	`Vec<Update>`	Balance changes for sync
`peer_height`	`u64`	Peer's current height
`merkle_root`	`[u8; 32]`	Integrity verification hash
`network_id`	`String`	Network ID for cross-check

Table 3: BlockPack Protocol Message Format

3.3.2 Concurrent Request Management

- **Request Pipelining**: Up to 3 concurrent block pack requests
- **Stale Request Detection**: Requests older than 35 seconds are cleared (just above libp2p's 30s timeout)
- **Adaptive Batch Sizing**: Batch size adjusts from 1,000 to 10,000 blocks based on network conditions, controlled by the Apollo PID system
- **LZ4 Compression**: 60–80% bandwidth reduction on block pack transfers

3.4 Dandelion++: Transaction Privacy

Q-NarwhalKnight implements Dandelion++ [8] for transaction propagation privacy, preventing network observers from linking transactions to originating IP addresses:

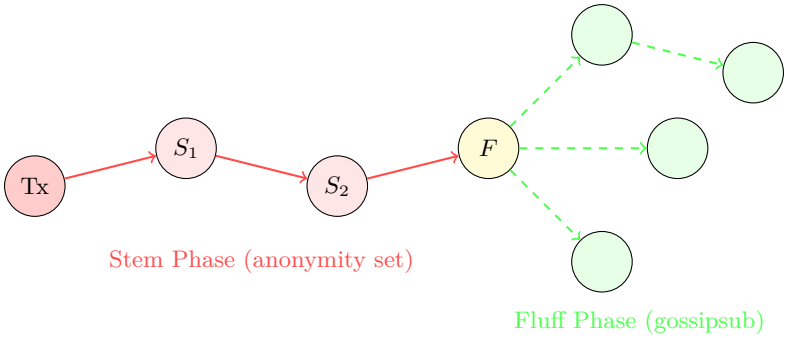


Figure 3: Dandelion++ Transaction Propagation

- **Stem Phase**: Transaction is forwarded along a single random path for 2–4 hops
- **Fluff Phase**: At a randomly selected node, the transaction enters normal gossipsub propagation
- **Fail-Safe**: If the stem relay fails or times out, the transaction automatically enters fluff mode after 30 seconds

4 Apollo Sync Optimization Engine

Apollo is Q-NarwhalKnight's adaptive synchronization engine, named for the Apollo space program's guidance systems. It provides real-time optimization of sync behavior through three integrated control systems.

4.1 Kalman Network Predictor

The Kalman filter maintains an optimal estimate of network conditions by fusing noisy observations from multiple peers:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}) \tag{1}$$

where the state vector $\mathbf{x} = [\text{bandwidth}, \text{latency}, \text{packet\_loss}]^T$ is estimated from peer observations $\mathbf{z}_k$.

State Variable	Unit	Description
Bandwidth	MB/s	Estimated available sync bandwidth
Latency	ms	Smoothed round-trip time to sync peers
Packet Loss	ratio	Estimated message loss probability
Confidence	0–1	Kalman filter confidence (innovation-based)

Table 4: Kalman Filter State Variables

The confidence metric drives the sync strategy:

- **LOCKED** (> 0.8): High confidence; use aggressive batch sizes
- **TRACKING** (0.5–0.8): Moderate confidence; standard parameters
- **ACQUIRING** (0.2–0.5): Building estimate; conservative settings
- **IDLE** (< 0.2): Insufficient data; use defaults

4.2 PID Rate Controller

The PID (Proportional-Integral-Derivative) controller regulates the block processing rate to match network capacity:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt} \tag{2}$$

where $e(t) = \text{target\_bps} - \text{actual\_bps}$ is the blocks-per-second error signal, and $u(t)$ is the batch size adjustment.

- **Target BPS**: Dynamically set based on Kalman bandwidth estimate
- **Drift Detection**: If actual BPS deviates $> 20\%$ from target, the controller adjusts batch sizes and concurrent request count
- **Anti-Windup**: Integral term is clamped to prevent oscillation during network congestion

4.3 Gravity Assist (Peer Momentum Model)

Inspired by gravitational slingshot maneuvers, the Gravity Assist system models peer "momentum" to optimize which peers to request blocks from:

$$\text{momentum}_i = \frac{\text{height}_i \cdot \text{response\_rate}_i}{\text{latency}_i^2} \tag{3}$$

- **Peer Momentum**: Higher-height, lower-latency peers have greater momentum
- **Cache Heat**: Tracks recently-accessed block ranges per peer ("hot" peers respond faster for adjacent ranges)
- **Gravitational Routing**: Sync requests are directed to the peer with highest momentum for the requested height range

The three Apollo systems work in concert: the Kalman predictor estimates the environment, the PID controller regulates throughput, and Gravity Assist optimizes peer selection.

5 DAG-Knight Consensus Integration

5.1 DAG Structure

Unlike linear blockchains, Q-NarwhalKnight blocks form a directed acyclic graph where each block references multiple parents:

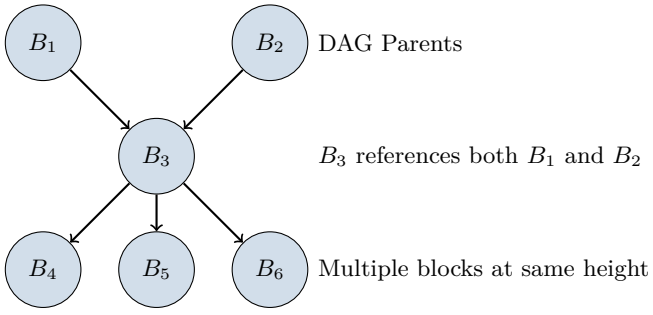


Figure 4: DAG Block Structure

5.2 Parent Selection

When producing a new block, miners select DAG parents based on:

1. **Recency**: Parents should be among the most recent blocks
2. **Coverage**: Parents should maximize coverage of the DAG tip set
3. **Availability**: Only reference parents that are locally available

5.3 Ordering and Finality

DAG-Knight achieves consensus ordering through a VDF-based anchor election mechanism:

1. Each epoch, validators compute a Verifiable Delay Function
2. The validator with the winning VDF output becomes the epoch anchor
3. The anchor's view of DAG ordering becomes canonical
4. Blocks referenced by the anchor achieve finality (typically < 3 seconds)

6 Narwhal Mempool Integration

6.1 Reliable Broadcast

Narwhal's reliable broadcast primitive ensures that if any correct node delivers a transaction, all correct nodes eventually deliver it. Q-NarwhalKnight implements this through:

1. **Primary Workers**: Batch transactions and create certificates
2. **Certificate Gossip**: Propagate certificates via gossipsub
3. **DAG Inclusion**: Include certificate hashes in block headers

6.2 Bullshark Finality

The Bullshark consensus protocol [7] runs atop the Narwhal DAG, providing:

- Optimistic fast path (2-round finality when leader is honest)
- Fallback path (4-round finality under Byzantine leader)

7 Synchronization Strategies

7.1 TurboSync: High-Speed Block Synchronization

For nodes significantly behind the network, Q-NarwhalKnight implements TurboSync, an Apollo-optimized synchronization protocol:

1. Node discovers it is behind via peer height announcements (peer_height $>$ local_height $+$ 5)
2. Apollo Kalman filter estimates available bandwidth and latency
3. Apollo PID controller calculates optimal batch size (1,000–10,000 blocks)

4. Apollo Gravity Assist selects the highest-momentum peer for the target range
5. Initiates parallel block pack requests (up to 3 concurrent)
6. Validates blocks in topological order (respecting DAG parents)
7. Commits blocks in LZ4-compressed batches for I/O efficiency
8. Updates height cache to actual contiguous height via database pointer

7.1.1 Height Cache Consistency

A critical invariant: the height cache must reflect the actual contiguous chain height, not the highest block received. After batch commit, the actual database pointer is read back:

```
// After batch commit, read actual pointer from database
let actual_height = storage.get_latest_qblock_height().await?;
storage.update_height_cache(actual_height).await;

// Persist safe floor with fsync for crash recovery
storage.save_safe_floor(max(contiguous_height, turbo_tip)).await;
```

Listing 2: Height cache consistency

7.2 Height Recovery After Restart

To prevent height regression on restart (a bug that caused nodes to roll back from height 5,431 to 2,502 in early mainnet), Q-NarwhalKnight maintains multiple recovery signals:

$$\text{recovery\_target} = \max(\text{highest\_block}, \text{db\_pointer}, \text{persisted\_tip}, \text{safe\_floor}) \tag{4}$$

The `safe_floor` is persisted with `fsync` every 15 seconds via the balance sync background task. The fast recovery scan threshold activates for any chain height above 1,000 blocks.

7.3 Gap Detection and Recovery

The network layer continuously monitors for chain gaps between the height cache and database pointer:

```
fn detect_gaps(&self) -> Option<(u64, u64)> {
    let cache_height = self.height_cache.cached();
    let pointer_height = self.get_qblock_latest_pointer()?;

    if cache_height > pointer_height + BATCH_SIZE {
        // Gap detected - cache raced ahead of actual chain
        Some((pointer_height, cache_height))
    } else {
        None
    }
}
```

Listing 3: Gap detection algorithm

7.4 Fork Detection and Reorganization

When receiving blocks at heights already occupied, Q-NarwhalKnight employs a fork-choice rule based on cumulative difficulty:

```rust
fn should_reorganize(&self, new_block: &QBlock, existing: &QBlock) -> bool {
    // Higher cumulative difficulty wins
    if new_block.cumulative_difficulty > existing.cumulative_difficulty {
        return true;
    }
    // Tie-breaker: lower hash value
    if new_block.cumulative_difficulty == existing.cumulative_difficulty {
        return new_block.hash < existing.hash;
    }
    false
}
```

Listing 4: Fork choice rule

8 Emission Economics and PI Controller

8.1 Emission Schedule

Q-NarwhalKnight uses a Bitcoin-inspired fixed-supply emission model with PI feedback control:

Parameter	Value	Notes
Maximum Supply	21,000,000 QUG	Hard cap, enforced at consensus level
Era 0 Emission	2,625,000 QUG/year	First 4 years
Halving Interval	4 years	Emission halves each era
Block Time	∼1 second	DAG-Knight target
Dev Fee	1%	Applied to mining rewards

Table 5: Emission Parameters

8.2 PI Emission Controller

To maintain the target emission rate despite variable block times and hash rate fluctuations, Q-NarwhalKnight employs a proportional-integral (PI) feedback controller:

$$\text{correction} = K_p \cdot (\text{target\_rate} - \text{actual\_rate}) + K_i \int (\text{target\_rate} - \text{actual\_rate})\, dt \tag{5}$$

The controller adjusts per-block rewards to converge on the target annual emission. As of v8.2.6 on mainnet, the PI correction factor is approximately 1.52×, indicating the controller is compensating for below-target block production by temporarily increasing per-block rewards.

8.3 Balance Persistence and Consensus

Critical Safety Property (v8.2.6): All balance updates are persisted to disk *immediately* via `put_sync` (fsync) after every transaction. This prevents a double-spend vulnerability discovered in v8.2.5 where balances were only synced every 15 seconds:

```rust
// After EVERY transfer - persist BOTH sender and receiver immediately
storage.save_wallet_balance(&tx.from, sender_new).await?;
storage.save_wallet_balance(&tx.to, receiver_new).await?;
// save_wallet_balance uses put_sync (fsync) - crash-durable
```

Listing 5: Immediate balance persistence (v8.2.6 fix)

Balance updates propagate through three independent code paths:

1. **Block Producer Path**: Local mining rewards and transactions
2. **DAG Gossipsub Path**: Blocks received from peers via gossipsub
3. **P2P Balance Gossipsub**: Direct balance state propagation

All three paths now use immediate persistence with fsync, ensuring no balance loss on node restart.

9 Security

9.1 Sync-Down Protection

A critical safety invariant prevents synchronizing to a lower height, which would catastrophically delete blocks:

```
// Application layer (main.rs)
if network_height > current_height + 5 {
    turbo_sync.sync_to_height(network_height).await;
}

// Database layer (turbo_sync.rs)
if target_height < local_height && local_height > 1000 {
    error!("CRITICAL: Sync-down blocked: {} -> {}", local_height, target_height
    );
    return Err(anyhow!("SAFETY ABORT: Refusing to sync down"));
}
```

Listing 6: Sync-down protection (multi-layer)

This defense operates at two independent layers—even if the application logic has a bug, the database layer will refuse to execute a sync-down operation.

9.2 Height-Gated Consensus Upgrades

All consensus rule changes are wrapped in block-height gates, ensuring historical blocks always validate with their original rules:

```
fn validate_signature(&self, block: &Block) -> Result<()> {
    if is_upgrade_active(Upgrade::PostQuantumSigs, block.height) {
        self.verify_dilithium_sig(block)?;  // New rule
    } else {
        self.verify_ed25519_sig(block)?;    // Historical rule
    }
    Ok(())
}
```

Listing 7: Height-gated consensus upgrade pattern

Activation heights are defined in advance and announced to the network, allowing nodes to upgrade binaries before activation without consensus disruption.

9.3 Eclipse Attack Mitigation

1. **Diverse Peer Selection**: Maintain connections to peers from different /16 subnets
2. **Bootstrap Pinning**: Always maintain at least one connection to a trusted bootstrap node (4 bootstrap nodes available)
3. **Outbound Connection Preference**: Prioritize self-initiated connections over incoming

9.4 Peer Trust and Reputation

Q-NarwhalKnight maintains per-peer trust scores based on block validity rate, response latency, bandwidth contribution, and protocol compliance. Peers falling below threshold are temporarily blacklisted.

9.5 Post-Quantum Cryptographic Agility

The architecture supports phased transition to post-quantum primitives:

- **Phase 0** (Current): Ed25519 signatures, QUIC/Noise transport
- **Phase 1** (Ready): Dilithium5 signatures, Kyber1024 KEM (height-gated activation)
- **Phase 2** (Planned): Hybrid Ed25519 + Dilithium5 during transition
- **Phase 3** (Future): Full post-quantum with QKD preparation layer

The crypto-agility layer allows algorithm substitution without protocol changes, activated via the height-gated upgrade system.

9.6 Pre-Flight Verification

Before serving requests after a binary upgrade, nodes optionally run a pre-flight verification:

1. Block chain integrity (blocks load and deserialize correctly)
2. Parent chain continuity (last 100 blocks have valid parent links)
3. Height pointer consistency (tip block exists and matches)
4. Schema version compatibility

This can be run in isolation (`Q_PREFLIGHT_ONLY=1`) to verify a new binary without affecting the live network.

10 High-Availability Deployment

10.1 Zero-Downtime Rolling Upgrades

All production deployments follow a six-step rolling upgrade pipeline that ensures zero user-visible downtime:

Step	Action	Description	Users Affected?
1	`verify-alpha`	SCP binary to Alpha Docker canary, verify	No
2	`verify-gamma`	SCP to Gamma, restart, health check	No
3	`promote`	Nginx: Gamma w=10, Beta w=1	No
4	Soak test	30s wait, verify Gamma handles traffic	No
5	`deploy-beta`	Stop Beta, replace, restart, verify	No
6	`restore`	Nginx: Beta w=10, Gamma w=1	No

Table 6: Rolling Upgrade Pipeline

Nginx uses `ip_hash` sticky sessions so each user consistently hits the same backend, preventing balance display flickering during transitions.

10.2 Automated Test Gate

The deployment script (`ha-deploy.sh`) runs 4,000+ automated tests before building, covering:

- Sync-down protection (prevents catastrophic data loss)
- Balance integrity (prevents incorrect balance display)
- Double-spend and replay attack prevention
- Fork detection and reorg safety
- DEX overflow protection (u128 arithmetic safety)
- Signature verification (prevents forged transactions)
- Block validation (prevents consensus failure)

Deployment is automatically aborted if any test fails.

11 Performance Characteristics

11.1 Production Metrics (Mainnet, February 2026)

Metric	Value	Conditions
Active Miners	182	Mainnet, 24h window
Aggregate Hash Rate	458 MH/s	Across all miners
Registered Wallets	394	Unique wallet addresses
Total Supply Mined	8,267.6 QUG	Of 21M maximum
Block Propagation Latency	< 50ms	4-server mesh
TurboSync Throughput	500–1,000 blocks/s	Apollo-optimized
Gossipsub Overhead	12%	Mesh factor $d = 6$
Connection Establishment	< 100ms	QUIC transport
DAG Finality	< 3 seconds	$3f + 1$ validators online
PI Emission Correction	1.52×	Self-correcting to target
Node Binary Size	∼84 MB	Statically linked, Linux x86_64

Table 7: Production Performance Metrics

11.2 Scalability

The network layer is designed to scale through:

- **Tiered Gossip**: Validators form a core gossip mesh; observers connect to validators
- **LZ4 Compression**: Block pack transfers achieve 60–80% bandwidth reduction
- **Apollo Adaptive Batching**: Automatically adjusts to network capacity
- **RocksDB Tuning**: Per-server block cache sizing (512MB to 4GB) prevents OOM while maximizing read throughput

12 Implementation Details

12.1 Technology Stack

- **Language**: Rust (100% safe code in networking layer)
- **Async Runtime**: Tokio with multi-threaded scheduler
- **libp2p Version**: 0.56.x with SwarmBuilder async pattern
- **Serialization**: Bincode for storage, CBOR for network messages
- **Database**: RocksDB with column family separation (Linux), Sled fallback (Windows)
- **Compression**: LZ4 for block packs, avoiding double-compression
- **AI Inference**: Optional llama-cpp integration for on-node AI chat (resource-throttled)
- **TUI**: Ratatui-based terminal dashboard with Apollo control system visualization

12.2 Crate Organization

```
crates/
+-- q-api-server/        # HTTP API, SSE streaming, block producer, mining
+-- q-network/           # UnifiedNetworkManager, gossipsub, Kademlia
|   +-- unified_network_manager.rs
|   +-- protocol_handshake.rs
|   +-- handshake_validator.rs
|   `-- auto_cluster.rs
+-- q-storage/           # RocksDB, TurboSync, balance consensus
|   +-- turbo_sync.rs
|   +-- balance_consensus.rs
|   +-- checkpoint_jumps.rs
|   `-- genesis_checkpoint.rs
+-- q-types/             # Block, Transaction, NetworkId definitions
+-- q-dandelion/         # Dandelion++ privacy layer
+-- q-dex/               # Decentralized exchange with AMM
+-- q-mining-pool/       # Distributed mining pool
+-- q-tui/               # Terminal UI with Apollo visualization
+-- q-miner/             # Standalone mining client
+-- q-quantum-crypto/    # Post-quantum cryptographic primitives
`-- q-consensus-guard/   # Height-gated upgrade system
```

12.3 Configuration

Key network parameters are configurable via environment variables and TOML:

```
[network]
bootstrap_peers = [
    "/ip4/185.182.185.227/tcp/9001/p2p/12D3KooWBHTC9Fhww...",
    "/ip4/109.205.176.60/tcp/9001/p2p/12D3KooWFqPX9TkvF4...",
    "/ip4/5.79.79.158/tcp/9001/p2p/12D3KooWQZZAyLA4VQmw...",
    "/ip4/161.35.219.10/tcp/9001/p2p/12D3KooWPwin4nJcU9P..."
]
max_peers = 50
gossipsub_mesh_n = 6
gossipsub_mesh_n_low = 4
```

```
gossipsub_mesh_n_high = 12
sync_batch_size = 5000        # Apollo PID adjusts dynamically
max_concurrent_sync = 3
stall_timeout_secs = 35
network_id = "mainnet-genesis"

# RocksDB tuning (per-server)
# ROCKSDB_BLOCK_CACHE_MB = 4096  (Beta, 16GB RAM)
# ROCKSDB_BLOCK_CACHE_MB = 512   (Gamma, 8GB RAM)
# ROCKSDB_BLOCK_CACHE_MB = 2048  (Delta)
```

Listing 8: Network configuration (production defaults)

13 Future Work

13.1 Completed Since v1.0 (Previously "Future Work")

- ✓ **Dandelion++ Integration**: Deployed in production for transaction privacy
- ✓ **Apollo Sync Engine**: Kalman, PID, and Gravity Assist operational
- ✓ **PI Emission Controller**: Self-correcting emission rate active on mainnet
- ✓ **4-Server HA Infrastructure**: Zero-downtime rolling deployments
- ✓ **Height-Gated Upgrades**: Consensus Guard system for safe rule changes
- ✓ **Mining Pool**: Distributed pool with stratum-compatible protocol
- ✓ **DEX**: On-chain decentralized exchange with AMM pools
- ✓ **TUI Dashboard**: Real-time Apollo control system visualization

13.2 In Progress

1. **Tor Integration**: Dedicated circuits per validator for IP anonymity (arti client)
2. **QRNG Circuit Seeding**: Quantum random number generation for Tor circuit selection
3. **Browser P2P**: Full WebRTC support for browser-native full nodes
4. **Cross-Shard Routing**: Protocol support for sharded execution
5. **Cross-Chain Bridges**: Bitcoin and Ethereum bridge protocols

13.3 Research Directions

- **Adaptive Mesh Optimization**: ML-based peer selection using Apollo telemetry
- **Recursive SNARK Weak Subjectivity**: Eliminating checkpoint trust assumptions
- **BEP44 DHT Records**: Fully decentralized peer discovery without bootstrap servers
- **Genus-2 Jacobian VDF Mining**: Novel proof-of-work based on algebraic geometry

14 Conclusion

Q-NarwhalKnight's network layer has evolved from a testnet prototype to a production mainnet sustaining 182 miners, 394 wallets, and continuous block production since February 22, 2026. The architecture demonstrates that combining modern peer-to-peer networking primitives (libp2p, gossipsub, Kademlia) with novel control-theoretic optimization (Apollo Kalman/PID/-Gravity) and rigorous safety engineering (sync-down protection, immediate balance persistence, height-gated upgrades) yields a system that is:

- **Performant**: Sub-second block propagation with Apollo-adaptive synchronization
- **Resilient**: Byzantine fault tolerant with automatic fork resolution across 4 bootstrap nodes
- **Extensible**: Crypto-agile design with height-gated upgrade path to post-quantum primitives
- **Economically Sound**: PI-controlled emission maintaining 2,625,000 QUG/year target with self-correcting feedback
- **Production-Hardened**: Zero-downtime deployment pipeline with 4,000+ automated safety tests

The unified network manager pattern, combined with the Apollo adaptive sync engine, provides a clean abstraction that separates protocol logic from transport concerns, enabling rapid iteration on consensus improvements without disrupting the networking layer.

References

[1] Danezis, G., Kokoris-Kogias, L., Sonnino, A., & Spiegelman, A. (2022). *Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus*. EuroSys 2022.

[2] Keidar, I., Kokoris-Kogias, E., Naor, O., & Spiegelman, A. (2021). *All You Need is DAG*. PODC 2021.

[3] Protocol Labs. (2024). *libp2p Specifications*. https://github.com/libp2p/specs

[4] Bernstein, D.J., & Lange, T. (2017). *Post-Quantum Cryptography*. Nature 549, pp. 188–194.

[5] Maymounkov, P., & Mazieres, D. (2002). *Kademlia: A Peer-to-peer Information System Based on the XOR Metric*. IPTPS 2002.

[6] Vyzovitis, D., et al. (2020). *GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks*. arXiv:2007.02754.

[7] Spiegelman, A., Giridharan, N., Sonnino, A., & Kokoris-Kogias, L. (2022). *Bullshark: DAG BFT Protocols Made Practical*. CCS 2022.

[8] Fanti, G., Venkatakrishnan, S.B., Bakshi, S., Denby, B., Bhargava, S., Miller, A., & Viswanath, P. (2018). *Dandelion++: Lightweight Cryptocurrency Networking with Formal Anonymity Guarantees*. ACM SIGMETRICS 2018.

[9] Kalman, R.E. (1960). *A New Approach to Linear Filtering and Prediction Problems*. Journal of Basic Engineering, 82(1), pp. 35–45.

[10] Facebook. (2024). *RocksDB: A Persistent Key-Value Store*. https://rocksdb.org

A Message Formats

A.1 Block Announcement (Gossipsub)

Field	Type	Size
Header		
height	`u64`	8 bytes
timestamp	`i64`	8 bytes
prev_block_hash	`[u8; 32]`	32 bytes
solutions_root	`[u8; 32]`	32 bytes
state_root	`[u8; 32]`	32 bytes
network_id	`String`	variable
dag_parents	`Vec<VertexId>`	variable
mining_solutions	`Vec<MiningSolution>`	variable
transactions	`Vec<Transaction>`	variable
Quantum Metadata		
qrng_entropy	`[u8; 32]`	32 bytes
pq_signature	`Option<Vec<u8»`	variable

Table 8: Block Announcement Message Format

A.2 Peer Height Announcement

Field	Type	Size
peer_id	`PeerId`	38 bytes
height	`u64`	8 bytes
block_hash	`[u8; 32]`	32 bytes
timestamp	`u64`	8 bytes
signature	`[u8; 64]`	64 bytes
network_id	`String`	variable
version	`String`	variable

Table 9: Peer Height Announcement Message Format (v8.x)

B Sync State Machine

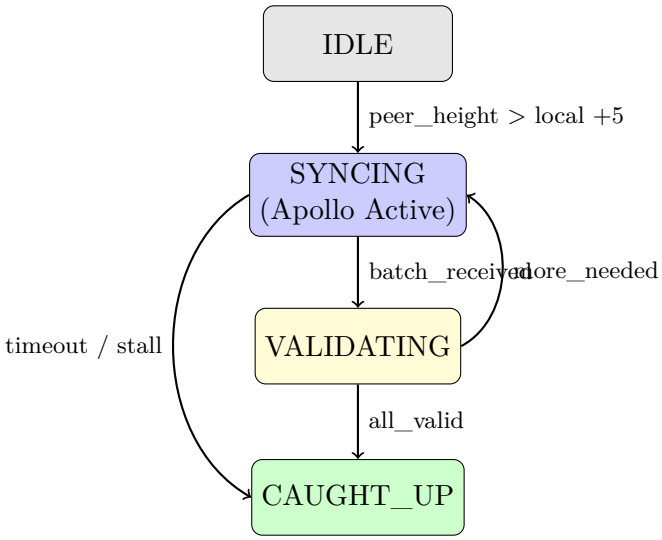


Figure 5: Synchronization State Machine

C Mainnet Network Statistics

Production statistics as of February 24, 2026:

Metric	**Value**
Network ID	`mainnet-genesis`
Software Version	v8.2.6
Bootstrap Nodes	4 (Beta, Gamma, Delta, Alpha)
Active Miners (24h)	182
Aggregate Hash Rate	458 MH/s
Registered Wallets	394
Total QUG Mined	8,267.6
Maximum Supply	21,000,000 QUG
Annual Emission (Era 0)	2,625,000 QUG/year
PI Correction Factor	1.52×
Era 0 Duration	4 years
Mining Health	Healthy

Table 10: Mainnet Statistics Snapshot (Feb 24, 2026)

Document generated for Q-NarwhalKnight v8.2.6 — Mainnet